Smart Pantry

High Level Design Document

Patrice Jones

CSMS Degree Program

Full Sail University

May 15, 2026

FULL SAIL UNIVERSITY

Table of Contents

Introduction

Project Concept / Pitch

Your pantry thinks ahead—reducing food waste through intelligent meal recommendations, expiration tracking, and AI-driven optimization.

Technical Design Goal(s)

The main technical goal of SmartPantry is to create an intelligent pantry and meal recommendation platform that helps users make better decisions with the food they already have at home. Instead of functioning like a traditional recipe app, the system is designed to analyze pantry inventory, ingredient expiration dates, nutritional value, cooking time, and eventually user behavior to generate smarter and more practical meal recommendations. One of the most unique aspects of the project is the recommendation engine itself. The system uses a weighted scoring model to rank recipes based on multiple real-world factors instead of simply matching ingredients. This allows the platform to prioritize meals that help reduce food waste, improve ingredient usage, and simplify meal planning in a more intelligent way. As the project develops further, machine learning techniques such as ingredient similarity analysis and personalized recommendation behavior will also be integrated to improve recommendation accuracy and adaptability.

From a technical perspective, one of the biggest challenges is designing a system that combines multiple components into one cohesive platform. The project requires frontend dashboard development, database management, recommendation-system architecture, analytics processing, and future machine learning integration to work together efficiently. Another challenge is balancing intelligent recommendation logic with usability so the system remains both technically advanced and easy for users to understand.

Personally, this project represents a major step forward in my development experience because it is the first time I have designed and built a large-scale intelligent software system from concept to implementation. It has pushed me to think beyond basic application development and focus more on scalable architecture, recommendation optimization, analytics, and real-world problem solving.

Development APIs / Programming Languages

SmartPantry is primarily being developed using Python because of its flexibility, strong machine learning support, and ability to integrate well with analytics and recommendation systems. The frontend interface is built using Streamlit, which allows the application to display interactive dashboards, pantry management tools, recommendation pages, and analytics visualizations in a clean and user-friendly way. The current database system uses SQLite for local data storage and inventory management during development. As the project grows, the architecture is designed to support future migration to PostgreSQL for improved scalability and multi-user support. Pandas is being used for data processing, analytics, and recommendation calculations, while Plotly and Matplotlib are planned for analytics dashboards and data visualization features.

For the intelligent recommendation system and future machine learning integration, the project will use scikit-learn, specifically tools such as CountVectorizer and cosine similarity algorithms to compare pantry ingredients against recipe datasets and improve recommendation quality. NumPy will also be used to support data processing and scoring calculations. GitHub is being used for version control, repository management, and development tracking, while Visual Studio Code serves as the primary development environment. Git Bash is also used for repository management, dependency installation, and command-line operations throughout development. The project is also being designed to support future integration with third-party APIs such as Spoonacular and the USDA FoodData Central API. These APIs would allow SmartPantry to access larger recipe datasets, nutritional information, ingredient metadata, and food-related analytics to improve recommendation accuracy and expand the system's capabilities.

Development Standards / Cold-Start Procedures

The SmartPantry project is managed using GitHub for version control and collaborative development. The repository is structured using a modular folder system to separate frontend application logic, recommendation-engine functionality, datasets, and database operations for easier maintenance and scalability. Development is currently being completed in Visual Studio Code using Python 3.11 and Streamlit as the primary framework for the user interface and dashboard system. The project follows several core engineering standards throughout development. Functions and variables use consistent naming conventions to improve readability and maintainability, and the codebase is organized into separate modules to avoid unnecessary duplication and improve scalability. Database operations, recommendation logic, and frontend components are intentionally separated into different files and directories so future developers can easily understand the project structure and extend functionality without rewriting the entire application. Git commits are also used regularly to document development progress and maintain stable working versions of the system. To set up the project locally, a developer must first clone the GitHub repository and open the project folder in Visual Studio Code. After cloning the repository, a Python virtual environment should be created and activated to isolate project dependencies. Once the virtual environment is active, all required dependencies can be installed using the requirements.txt file. After dependencies are installed, the Streamlit application can be launched using the command:

```
streamlit run app/main.py
```

The project structure currently includes separate folders for the application interface, recommendation engine, datasets, and database systems. SQLite is used for local database storage during development, and the application automatically initializes required tables when launched. Future versions of the project are being designed with scalability in mind, including possible migration to PostgreSQL, integration of machine learning recommendation systems, and cloud deployment support.

Project Functionality

User Modes

SmartPantry currently supports a single-user mode where users can manage pantry inventory, track expiration dates, generate meal recommendations, and review analytics through one centralized dashboard. Within the application, users can switch between several functional modes including pantry management, recommendation generation, grocery planning, and analytics review. The pantry management mode allows users to add, edit, organize, and monitor ingredients stored within the system. The recommendation mode analyzes pantry inventory and generates ranked meal suggestions based on ingredient availability, expiration urgency, cooking time, and future personalization factors. Grocery planning mode helps users identify missing ingredients needed to complete recommended meals, while the analytics mode provides insights into ingredient usage, pantry efficiency, and food waste trends. Although the current MVP focuses on a single-user architecture, the system is being designed with future scalability in mind. Planned future support includes multi-user household functionality, collaborative pantry management, cloud synchronization, and personalized user profiles.

Main Mechanics / Sub Mechanics

The core mechanic of SmartPantry is the intelligent recommendation engine. The system compares pantry inventory against recipe datasets and generates meal recommendations using weighted scoring algorithms. Instead of relying only on simple ingredient matching, the recommendation system evaluates multiple factors including pantry availability, expiration urgency, nutritional optimization, cooking time, and user preferences to produce more practical and efficient meal suggestions. Several sub-mechanics support the recommendation engine. Ingredient normalization helps standardize pantry and recipe ingredient names for more accurate comparisons. Expiration analysis prioritizes recipes that use ingredients approaching expiration to help reduce food waste. Recommendation ranking calculates weighted scores to organize recipes based on practicality and efficiency. Grocery-list generation identifies missing ingredients needed to complete selected recipes, while analytics systems track pantry usage patterns and recommendation effectiveness.

The project also includes planned machine learning integration using ingredient vectorization and cosine similarity analysis to mathematically compare pantry inventory with recipe datasets. This approach was researched because it provides a realistic and scalable recommendation-system solution that fits within the scope of the project while still demonstrating graduate-level system intelligence. If advanced machine learning recommendation models become too resource-intensive or difficult to complete within the project timeline, the weighted recommendation-scoring system can still function independently as a reliable fallback recommendation architecture. Similar recommendation concepts are used in applications such as Samsung Food, SuperCook, and Yummly, although SmartPantry focuses more heavily on expiration optimization, analytics, and food waste reduction.

AI Module(s)

The AI component focuses on intelligent recommendation scoring, recommendation ranking, personalization, and future machine learning similarity analysis. The recommendation system is being designed around a multi-factor scoring model that evaluates pantry ingredients, expiration urgency, cooking time, nutritional value, and user preferences together instead of relying on basic ingredient matching alone. Future machine learning integration will use scikit-learn tools such as CountVectorizer and cosine similarity algorithms to mathematically compare pantry inventory against recipe datasets and improve recommendation quality. One of the main reasons for selecting this approach is that it provides explainable recommendation logic while still introducing measurable machine learning concepts into the project. The architecture also supports future expansion into adaptive recommendation systems and personalized recommendation learning. The largest risks associated with the AI component involve recommendation quality, ingredient normalization accuracy, and balancing intelligent recommendation complexity with application performance and usability. There is also a development risk related to scope management, since

more advanced machine learning systems can quickly increase implementation complexity. To manage this risk, the project is being developed in phases so the weighted recommendation engine can function independently before additional AI layers are added. Yes, SmartPantry will incorporate AI-assisted recommendation functionality as one of the project's primary technical features.

User Interface / Inputs / Sensors

The SmartPantry interface is being developed using Streamlit and focuses on a clean, dashboard-style experience that allows users to navigate between pantry management, recommendation generation, grocery planning, and analytics views. The interface is designed to remain simple and intuitive while still supporting advanced recommendation and analytics functionality. Users currently interact with the application using standard keyboard and mouse input through forms, dropdown menus, buttons, filters, and dashboard controls. The interface supports ingredient entry, inventory updates, recommendation viewing, and analytics interaction directly within the application dashboard. The project does not currently require specialized hardware sensors or external peripherals during the MVP phase. However, future expansion plans include barcode scanning, OCR receipt scanning, and mobile-friendly interfaces to improve inventory input efficiency and user convenience.

Graphics / Animation / Video

The current version of SmartPantry primarily uses dashboard-style visualizations, analytics charts, recommendation cards, and structured user-interface elements rather than advanced animation or video systems. Plotly and Matplotlib are planned for analytics visualization and reporting features, including pantry utilization charts, expiration tracking graphs, and recommendation analytics dashboards. Because the project is centered around data analysis and recommendation systems rather than multimedia presentation, advanced rendering systems, shaders, or animation frameworks are not currently required. The visual focus of the platform is usability, readability, and analytics clarity rather than cinematic graphics or custom rendering technology. Future versions may incorporate food imagery, recommendation thumbnails, and enhanced UI styling to improve user experience and visual engagement.

Audio

The current version of SmartPantry primarily uses dashboard-style visualizations, analytics charts, recommendation cards, and structured user-interface elements rather than advanced animation or video systems. Plotly and Matplotlib are planned for analytics visualization and reporting features, including pantry utilization charts, expiration tracking graphs, and recommendation analytics dashboards. Because the project is centered around data analysis and recommendation systems rather than multimedia presentation, advanced rendering systems, shaders, or animation frameworks are not currently required. The visual focus of the platform is usability, readability, and analytics clarity rather than cinematic graphics or custom rendering technology. Future versions may incorporate food imagery, recommendation thumbnails, and enhanced UI styling to improve user experience and visual engagement.

Networking

The current MVP version of SmartPantry primarily operates as a local application and does not require advanced networking architecture during the initial development phase. SQLite is currently used for local database storage and recommendation processing. However, the project architecture is intentionally designed to support future networking functionality including cloud deployment, collaborative household synchronization, API integration, and multi-user support. Future backend networking plans may include FastAPI services, REST API communication, PostgreSQL cloud database integration, and third-party API connectivity. Potential third-party services include the Spoonacular API and USDA FoodData Central API for recipe datasets, nutritional information, and ingredient metadata. Future networking functionality would primarily transfer pantry data, recipe metadata, recommendation scores, user preferences, and analytics information between client dashboards and cloud-hosted backend systems. Because the current MVP is locally hosted, packet synchronization and network reliability concerns are minimal during the current phase

of development. However, future cloud-based versions would require proper API request handling, session management, database synchronization, and reliable backend communication systems to support multi-user functionality and real-time updates.

Databases / Server-Side Technology

Yes, SmartPantry requires a database system to function because the application depends on persistent pantry inventory storage, recommendation processing, analytics tracking, and future personalization features. The current MVP version uses SQLite as the primary local database solution because it is lightweight, easy to integrate with Python, and efficient for early-stage development and testing. The database currently stores pantry inventory information such as ingredient names, quantities, expiration dates, categories, and future recommendation-history data. SQLite was selected for the MVP because it allows rapid development without requiring a separate database server while still supporting structured relational data management. Database integration has already been implemented using Python and SQL queries, and the application automatically initializes required database tables during startup.

The project architecture is also being designed with future scalability in mind. As SmartPantry evolves, the database system may transition to PostgreSQL to support cloud deployment, multi-user functionality, collaborative household systems, and larger recommendation datasets. Research has already been conducted on scalable database integration, relational schema organization, recommendation-history tracking, and API-based backend communication to support future server-side expansion. Future server-side development may also include FastAPI integration for backend services, cloud-hosted databases, and API communication between frontend dashboards and recommendation systems.

Localization Plan

The MVP version of SmartPantry is currently focused on English-language support for users in the United States. Since the project's initial focus is recommendation-system functionality, analytics, and pantry optimization, localization is considered a secondary development priority during the early phases of development. However, the system architecture is intentionally designed to support future localization expansion. Future versions may include multilingual support, international measurement conversions, region-specific grocery terminology, and localized recipe datasets depending on the geographic region being supported. This would allow the application to better accommodate different food cultures, ingredient naming conventions, and nutritional standards. Most of the text within the project exists in the user interface, dashboard navigation, recommendation descriptions, analytics labels, and future grocery-planning systems. If localization is implemented, translated text will likely be stored in structured configuration files or database tables to allow easier language switching and future scalability without modifying the core application logic. At the current stage of development, no major cultural modifications are required because the MVP remains focused on core recommendation-system functionality rather than global deployment.

Custom Topics

One of the most important custom aspects of SmartPantry is its focus on intelligent food optimization rather than simple recipe discovery. The project combines pantry management, expiration tracking, recommendation scoring, analytics, and future machine learning integration into one unified recommendation platform. This combination of systems creates a more intelligent and practical meal-planning experience compared to traditional recipe applications. Several future expansion features are also being considered beyond the MVP scope. One planned feature is barcode scanning integration, which would allow users to quickly add pantry items by scanning product packaging using a mobile device camera. Another planned enhancement is OCR receipt scanning, where grocery receipts could be analyzed automatically to populate pantry inventory without manual entry. These features would improve usability while reducing user input time.

Additional future concepts include predictive grocery forecasting, personalized recommendation learning, collaborative household pantry systems, nutrition optimization tools, and cloud synchronization across devices. The recommendation system architecture is intentionally modular so these features can be

added gradually without requiring major redesign of the existing platform. While many of these advanced systems are not required for the MVP, the current architecture and research direction already support future integration and scalability, which is an important part of the project's long-term design strategy.

Mockups / Flowcharts

The SmartPantry application is designed with a clean dashboard-style interface focused on usability, accessibility, and real-time interaction between pantry inventory management and intelligent recipe recommendations. The interface follows a multi-page navigation structure that allows users to move between the Dashboard, Pantry Management, and Smart Recommendation modules seamlessly.

The primary navigation menu is positioned on the left-hand sidebar of the application and includes:
- Dashboard
- Pantry
- Recommendations

This sidebar design allows users to quickly switch between core application features while maintaining a consistent user experience throughout the platform.

The main Dashboard screen serves as the application's control center. It displays high-level analytics such as:
- Total pantry items
- Total available recipes
- Current pantry inventory table
- Real-time recommendation previews

The Pantry Inventory screen allows users to:
- Add pantry items
- Edit existing inventory
- Update quantities
- Modify expiration dates
- Categorize ingredients
- View stored pantry data in tabular form

The Smart Recommendation interface dynamically analyzes pantry inventory data against recipe datasets and presents recommended meals based on ingredient compatibility and scoring metrics. Each recommendation card includes:
- Recipe title
- Ingredient list
- Cook time
- Recommendation score
- Pantry match status
- Missing ingredient analysis

Application Flow

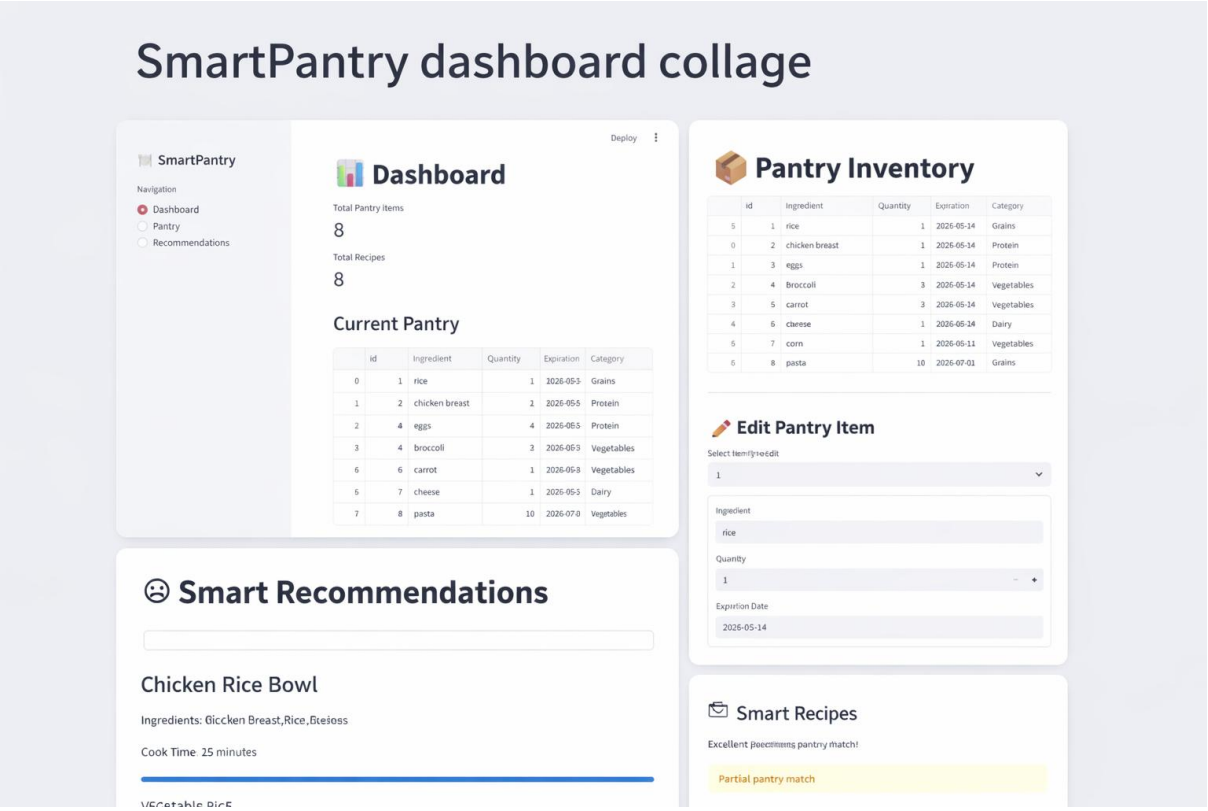
1. User launches the SmartPantry application
2. User navigates to the Pantry page
3. User adds or edits pantry inventory items
4. Pantry data is stored in the SQLite database
5. Recommendation engine evaluates pantry inventory against recipe datasets
6. Recommendation scores are calculated using inventory matching and expiration scoring logic
7. Recommended meals are displayed to the user
8. User reviews recommendation analytics and pantry compatibility results

Interface Design Goals

The SmartPantry interface was designed with the following goals:
• Minimal learning curve for users
• Clean dashboard-style presentation
• Fast inventory management workflow
• Real-time recommendation feedback
• Expandable architecture for future AI and analytics integration
• Mobile-friendly and scalable layout design

The current implementation uses Streamlit for rapid UI development and interactive dashboard functionality. Future iterations may include advanced visual analytics dashboards, machine learning recommendation visualizations, and user personalization controls.

Current SmartPantry UI Mockup



Production Documentation

Feature List

Number	Feature	Category	Priority
F1	Interactive Dashboard Analytics	UI	High

F2	Pantry Inventory Management	System	High
F3	Add Pantry Items	UI	High
F4	Edit Pantry Items	UI	High
F5	Pantry Expiration Tracking	System	High
F6	Recipe Recommendation Engine	System	High
F7	Pantry-to-Recipe Matching Logic	System	High
F8	Recommendation Scoring System	System	High
F9	Ingredient Compatibility Analysis	System	Medium
F10	Missing Ingredient Detection	System	Medium
F11	SQLite Database Integration	System	High
F12	Real-Time Recommendation Updates	System	Medium
F13	Streamlit Web Interface	Tool	High
F14	Python Application Development	Tool	High
F15	Pandas Data Processing	Tool	High
F16	Recommendation Dataset Management	System	Medium
F17	CRUD Database Operations	System	High
F18	Recipe Dataset CSV Integration	System	Medium
F19	Inventory Category Classification	System	Medium
F20	Recommendation Ranking Algorithm	System	High
F21	Machine Learning Recommendation Expansion	System	High
F22	Personalized User Recommendation Profiles	System	Medium

F23	Predictive Expiration Analytics	System	Medium
F24	Food Waste Reduction Metrics	System	Medium
F25	Data Visualization Components	UI	Medium
F26	Recommendation Confidence Scoring	System	Medium
F27	Smart Recipe Filtering	UI	Medium
F28	User-Friendly Navigation Sidebar	UI	High
F29	Recommendation Performance Evaluation	System	Medium
F30	Modular Python Architecture	Tool	High
F31	GitHub Version Control Integration	Tool	High
F32	Virtual Environment Management	Tool	High
F33	AI-Driven Recommendation Optimization	System	Medium
F34	Future Cloud Deployment Support	System	Low
F35	Future Multi-User Authentication Support	System	Low
F36	Smart Recommendation Explanation Engine	System	Medium
F37	Recommendation Feedback Collection	UI	Medium
F38	Experimental Analytics and Evaluation Module	System	Medium
F39	Intelligent Pantry Health Scoring	System	Medium
F40	Recommendation Accuracy Tracking	System	Medium

Master Task List

Number	Task	Feature	Dependencies
T1	Design dashboard page layout	F1	NA
T2	Develop pantry inventory database schema	F2	NA
T3	Implement pantry item add functionality	F3	T2
T4	Implement pantry item edit functionality	F4	T2
T5	Develop expiration tracking logic	F5	T2
T6	Design recipe recommendation engine architecture	F6	NA
T7	Implement pantry-to-recipe matching algorithm	F7	T6
T8	Develop recommendation scoring calculations	F8	T7
T9	Implement ingredient compatibility analysis	F9	T7
T10	Develop missing ingredient detection logic	F10	T7
T11	Configure SQLite database integration	F11	T2
T12	Implement real-time recommendation refresh system	F12	T8
T13	Create Streamlit dashboard interface	F13	T1
T14	Configure Python virtual environment and dependencies	F14	NA
T15	Integrate Pandas data processing workflows	F15	T11
T16	Create recipe dataset	F16	T15

	management structure		
T17	Develop CRUD database operations	F17	T11
T18	Import and validate recipe CSV dataset	F18	T16
T19	Implement ingredient category classification	F19	T2
T20	Develop recommendation ranking logic	F20	T8
T21	Research machine learning recommendation models	F21	T20
T22	Develop personalized recommendation profile system	F22	T21
T23	Design predictive expiration analytics module	F23	T5
T24	Implement food waste reduction metrics	F24	T23
T25	Develop dashboard data visualizations	F25	T1
T26	Create recommendation confidence calculations	F26	T20
T27	Implement smart recipe filtering interface	F27	T20
T28	Design sidebar navigation workflow	F28	T13
T29	Develop recommendation evaluation metrics	F29	T20

Development Roadmap

Milestone 1 — Foundation and Core System Development

T1 – Configure Development Environment and Repository Structure
Set up the SmartPantry project architecture, including Python virtual environment configuration, GitHub repository integration, dependency management, and modular folder organization. Establish coding standards, naming conventions, and development workflows to support long-term scalability and collaboration.
T2 – Develop SQLite Database and Pantry Management System
Design and implement the SQLite database schema for pantry inventory storage. Create CRUD functionality for adding, viewing, editing, and updating pantry items. Validate database communication and ensure persistent local storage functionality.
T3 – Create Initial Streamlit User Interface
Develop the foundational Streamlit dashboard interface, including sidebar navigation, pantry inventory displays, and application routing between major pages such as Dashboard, Pantry, and Recommendations.
T4 – Integrate Recipe Dataset and CSV Processing
Import and validate recipe datasets using Pandas. Build preprocessing logic for ingredient parsing, category handling, cook time processing, and dataset normalization to prepare the recommendation engine for analysis.

Milestone 2 — Recommendation Engine and Intelligent Functionality

T5 – Develop Pantry-to-Recipe Matching Algorithm
Implement the core recommendation engine responsible for comparing pantry inventory against recipe ingredients. Build ingredient matching logic and identify compatible, partial, and incompatible recipes.
T6 – Develop Recommendation Scoring System
Design a weighted recommendation scoring system based on pantry inventory availability, expiration priority, ingredient compatibility, and missing ingredient penalties. Create ranking functionality to prioritize higher-quality recommendations.
T7 – Implement Expiration Awareness and Food Waste Reduction Logic
Create expiration tracking calculations that prioritize recipes using ingredients nearing expiration. Integrate food waste reduction analytics into recommendation calculations and dashboard reporting.
T8 – Build Dynamic Recommendation Interface
Develop recommendation cards and analytics panels displaying recommendation scores, missing ingredients, cook time estimates, pantry match percentages, and recommendation confidence levels within the Streamlit interface.

Milestone 3 — Advanced Analytics and Graduate-Level Enhancements

T9 – Develop Recommendation Evaluation and Analytics Metrics
Implement measurable analytics to evaluate recommendation quality, including recommendation accuracy, pantry utilization rates, expiration reduction metrics, and user interaction tracking.
T10 – Research and Prototype Machine Learning Integration
Research machine learning approaches for intelligent personalization and predictive recommendation optimization. Prototype scalable AI modules capable of improving recommendation quality over time using historical pantry behavior and recommendation trends.
T11 – Develop Personalized Recommendation Expansion Features
Begin implementation of personalized recommendation profiles, user preference modeling, ingredient prioritization, and adaptive recommendation filtering systems to improve recommendation relevance and usability.

T12 – Optimize User Interface and Finalize System Integration
Refine dashboard layouts, improve usability and responsiveness, optimize database queries, finalize application integration, and conduct system-wide testing, debugging, and validation for capstone presentation readiness.

Milestone 4 — Finalization, Testing, and Capstone Delivery

T13 – Conduct Verification and Validation Testing
Perform comprehensive testing across all modules, including database operations, recommendation logic, scoring calculations, user interface interactions, and performance validation under multiple pantry scenarios.
T14 – Complete Technical Documentation and Capstone Deliverables
Finalize all project documentation including the High-Level Design Document, architecture diagrams, flowcharts, testing documentation, evaluation metrics, and development summaries aligned with graduate-level capstone requirements.
T15 – Prepare Final Demonstration and Presentation Materials
Develop demonstration workflows, presentation slides, screenshots, analytics summaries, and technical explanations showcasing SmartPantry's intelligent recommendation capabilities, measurable results, and future scalability potential.

Verification and Validation (V&V)

Task	Verification	Validation
T1 – Configure Development Environment and Repository Structure	The repository successfully clones from GitHub, dependencies install without errors, and the Streamlit application launches correctly using the project startup command.	Developers can follow setup instructions without confusion and begin contributing to the project with minimal setup issues.
T2 – Develop SQLite Database and Pantry Management System	Pantry items are successfully inserted, updated, retrieved, and stored persistently within the SQLite database tables. Database queries return accurate inventory information without duplication or corruption.	Users can reliably add and manage pantry inventory without losing data between application sessions.
T3 – Create Initial Streamlit User Interface	Sidebar navigation correctly routes users between Dashboard, Pantry, and Recommendations pages without rendering or routing errors.	Users can easily understand how to navigate the application and locate major features without additional guidance.
T4 – Integrate Recipe Dataset and CSV Processing	Recipe datasets load successfully into Pandas DataFrames and ingredient data is parsed correctly without formatting failures or missing records.	Recipe information appears readable, organized, and relevant to the user during recommendation generation.
T5 – Develop Pantry-to-Recipe Matching Algorithm	Pantry ingredients are correctly compared to recipe ingredients and matching recipes are identified consistently across multiple inventory scenarios.	Users receive meal recommendations that accurately reflect ingredients currently stored in their pantry.
T6 – Develop Recommendation Scoring System	Recommendation scores are calculated correctly using weighted scoring factors including ingredient matches, expiration priority, and missing ingredient analysis. Recipes are ranked consistently according to scoring logic.	Users perceive higher-ranked recommendations as more practical and useful than lower-ranked recommendations.
T7 – Implement Expiration Awareness and Food Waste Reduction Logic	Ingredients nearing expiration increase recommendation priority according to expiration scoring calculations. Expiration tracking updates correctly based on current date values.	Users can clearly identify which recommendations help reduce potential food waste.
T8 – Build Dynamic Recommendation Interface	Recommendation cards correctly display recipe names, ingredient lists, recommendation scores, missing ingredients, and pantry match information without formatting issues.	Users can quickly understand recommendation details and make meal decisions without confusion.
T9 – Develop Recommendation Evaluation and Analytics Metrics	Pantry utilization calculations, recommendation metrics, and analytics visualizations display	Users find analytics dashboards informative and useful for understanding pantry usage and

	mathematically correct values based on current inventory data.	recommendation effectiveness.
T10 – Research and Prototype Machine Learning Integration	Ingredient vectorization and cosine similarity calculations generate recommendation similarity scores without runtime or processing failures.	Recommendation quality appears more personalized and intelligent compared to basic ingredient matching alone.
T11 – Develop Personalized Recommendation Expansion Features	User preference data is stored correctly and recommendation rankings adjust based on stored preference information.	Users feel recommendations better reflect their cooking habits and ingredient preferences over time.
T12 – Optimize User Interface and Finalize System Integration	All application modules operate together without routing failures, broken dependencies, or database synchronization issues.	Users experience smooth interaction between pantry management, recommendations, and analytics workflows.
T13 – Conduct Verification and Validation Testing	All major application systems pass testing scenarios without critical errors, crashes, or incorrect recommendation calculations.	Users can complete major workflows successfully without encountering blocking issues or usability confusion.
T14 – Complete Technical Documentation and Capstone Deliverables	Documentation accurately reflects implemented system architecture, recommendation logic, feature functionality, and development workflows.	Readers can understand the project scope, technical implementation, and research direction clearly through the provided documentation.
T15 – Prepare Final Demonstration and Presentation Materials	Demonstration materials accurately represent implemented features, analytics, recommendation logic, and project functionality.	Viewers can clearly understand the purpose, technical depth, and intelligent functionality of SmartPantry during presentations and demonstrations.

References

[1] United States Department of Agriculture, "Food Loss and Waste," USDA, 2025. [Online]. Available: https://www.usda.gov/about-food/food-safety/food-loss-and-waste. [Accessed: May 15, 2026].
[2] United States Environmental Protection Agency, "Food Waste and Food Recovery," EPA, 2025. [Online]. Available: https://www.epa.gov/sustainable-management-food/food-recovery-hierarchy. [Accessed: May 15, 2026].
[3] Samsung Food, "Samsung Food Platform," 2025. [Online]. Available: https://food.samsung.com/. [Accessed: May 15, 2026].
[4] SuperCook, "Recipe Search by Ingredients," 2025. [Online]. Available: https://www.supercook.com/. [Accessed: May 15, 2026].
[5] Scikit-learn Developers, "scikit-learn Machine Learning Library," 2025. [Online]. Available: https://scikit-learn.org/. [Accessed: May 15, 2026].
[6] Streamlit Inc., "Streamlit Documentation," 2025. [Online]. Available: https://docs.streamlit.io/. [Accessed: May 15, 2026].
[7] Pandas Development Team, "Pandas Documentation," 2025. [Online]. Available: https://pandas.pydata.org/docs/. [Accessed: May 15, 2026].
[8] Spoonacular, "Food and Recipe API," 2025. [Online]. Available: https://spoonacular.com/food-api. [Accessed: May 15, 2026].
[9] U.S. Department of Agriculture, "FoodData Central API Guide," 2025. [Online]. Available: https://fdc.nal.usda.gov/api-guide.html. [Accessed: May 15, 2026].